T4 Actividades de redes neuronales 2

2 Algoritmo backprop

☐ El algoritmo backprop es la técnica estándar para el entrenamiento de redes. En relación con el mismo, indica la afirmación incorrecta o escoge la última opción si las tres primeras son correctas:

1. Backprop calcula el gradiente de la pérdida con respecto a los parámetros de cada capa y lo pasa a un algoritmo de optimización.
2. Su aplicación al entrenamiento de perceptrones multicapa, de la manera que ha pasado a ser estándar, se atribuye a Paul Werbos, en 1982.
3. Su popularización se debe en gran medida a un artículo de Rumelhart, Hinton y Williams en Nature, en 1986.
4. Las tres afirmaciones anteriores son correctas.

Solución: La 4.

☐ El algoritmo backprop ejecuta dos pasos básicos (por iteración): forward y backward. En relación con los mismos, indica la afirmación incorrecta o escoge la última opción si las tres primeras son correctas:

1. Forward recorre el modelo hacia adelante para hallar la salida de cada capa a partir de una entrada.
2. Backward recorre el modelo hacia atrás para hallar el gradiente de la pérdida respecto a los parámetros
3. Primero se ejecuta el paso forward y luego el backward.
4. Las tres afirmaciones anteriores son correctas.

Solución: La 4.

3 Ejemplo de regresión con un dato

Problema. Sea el MLP

$$\boldsymbol{x}^{(1)} \in \mathbb{R}^3 \;\rightarrow\; \boldsymbol{x}^{(2)} = \mathbf{W}^{(1)}\boldsymbol{x}^{(1)} + \boldsymbol{b}^{(1)} \in \mathbb{R}^2 \;\rightarrow\; \boldsymbol{x}^{(3)} = \boldsymbol{\sigma}(\boldsymbol{x}^{(2)}) \in \mathbb{R}^2 \;\rightarrow\; x^{(4)} = \mathbf{W}^{(3)}\boldsymbol{x}^{(3)} + b^{(3)} \in \mathbb{R}$$

$$\mathbf{W}^{(1)} = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix} \qquad \boldsymbol{b}^{(1)} = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \qquad \mathbf{W}^{(3)} = (1, 1) \qquad b^{(3)} = 1$$

Dado $(\boldsymbol{x}, y) = (\mathbf{1}, 1)$, se está aplicando backprop para minimizar la pérdida cuadrática mediante una iteración de GD con $\eta = 0.1$. Se ha completado el paso forward:

$$\boldsymbol{x}^{(1)} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} \;\rightarrow\; \boldsymbol{x}^{(2)} = \begin{pmatrix} 4 \\ 4 \end{pmatrix} \;\rightarrow\; \boldsymbol{x}^{(3)} = \begin{pmatrix} 0.982 \\ 0.982 \end{pmatrix} \;\rightarrow\; \hat{y} = x^{(4)} = 2.9640 \;\rightarrow\; \mathcal{L} = x^{(5)} = 3.8574$$

Aplica el paso backward (parcialmente) y actualiza $\mathbf{W}^{(3)}$.

Solución:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = 2(\hat{y} - y) = 3.9280$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(3)}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial \mathbf{W}^{(3)}} = \boldsymbol{x}^{(3)} \frac{\partial \mathcal{L}}{\partial \hat{y}} = \begin{pmatrix} 3.8574 \\ 3.8574 \end{pmatrix}$$

$$\mathbf{W}^{(3)} = \mathbf{W}^{(3)} - \eta \left(\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(3)}} \right)^t = (1, 1) - 0.1\,(3.8574, 3.8574) = (0.6143, 0.6143)$$

In [ ]:
```
import numpy as np; np.set_printoptions(precision=4); import keras
x1 = keras.Input(shape=(3,)); C1 = keras.initializers.Constant(1.)
x3 = keras.layers.Dense(2, activation='sigmoid', kernel_initializer=C1, bias_initializer=C1)(x1)
hy = keras.layers.Dense(1, activation=None, kernel_initializer=C1, bias_initializer=C1)(x3)
M = keras.Model(inputs=x1, outputs=hy)
```

In [ ]:
```
X = np.array([[1, 1, 1]]); y = np.array([[1]])
M_x3 = keras.Model(inputs=x1, outputs=M.layers[1].output); print('x3 =', M_x3(X))
print('hy =', M(X))
M.compile(loss='mean_squared_error', optimizer=keras.optimizers.SGD(learning_rate=0.1))
M.fit(X, y, epochs=1, verbose=1); print(M.layers[1].get_weights(), "\n", M.layers[2].get_weights())
```

```
x3 = tf.Tensor([[0.982 0.982]], shape=(1, 2), dtype=float32)
hy = tf.Tensor([[2.964]], shape=(1, 1), dtype=float32)
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 287ms/step - loss: 3.8574
[array([[0.9931, 0.9931],
       [0.9931, 0.9931],
       [0.9931, 0.9931]], dtype=float32), array([0.9931, 0.9931], dtype=float32)]
 [array([[0.6143],
       [0.6143]], dtype=float32), array([0.6072], dtype=float32)]
```

4 Ejemplo de regresión con un batch

Problema. Sea el MLP

$$\boldsymbol{x}^{(1)} \in \mathbb{R}^3 \;\rightarrow\; \boldsymbol{x}^{(2)} = \mathbf{W}^{(1)}\boldsymbol{x}^{(1)} + \boldsymbol{b}^{(1)} \in \mathbb{R}^2 \;\rightarrow\; \boldsymbol{x}^{(3)} = \boldsymbol{\sigma}(\boldsymbol{x}^{(2)}) \in \mathbb{R}^2 \;\rightarrow\; x^{(4)} = \mathbf{W}^{(3)}\boldsymbol{x}^{(3)} + b^{(3)} \in \mathbb{R}$$

$$\mathbf{W}^{(1)} = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix} \qquad \boldsymbol{b}^{(1)} = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \qquad \mathbf{W}^{(3)} = (1, 1) \qquad b^{(3)} = 1$$

Dado $\mathbf{X} = \begin{pmatrix} \boldsymbol{x}_1^t \\ \boldsymbol{x}_2^t \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \end{pmatrix}$ y $\boldsymbol{y} = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, se está aplicando backprop para minimizar la pérdida cuadrática mediante una iteración de GD con $\eta = 0.1$. Se ha completado el paso forward:

$$\mathbf{X}^{(1)} = \mathbf{X} \;\rightarrow\; \mathbf{X}^{(2)} = \begin{pmatrix} 4 & 4 \\ 7 & 7 \end{pmatrix} \;\rightarrow\; \mathbf{X}^{(3)} = \begin{pmatrix} 0.9820 & 0.9820 \\ 0.9991 & 0.9991 \end{pmatrix} \;\rightarrow\; \hat{\boldsymbol{y}} = \mathbf{X}^{(4)} = \begin{pmatrix} 2.9640 \\ 2.9982 \end{pmatrix} \;\rightarrow\; \mathcal{L} = x^{(5)} = 2.4268$$

Aplica el paso backward (parcialmente) y actualiza $\mathbf{W}^{(3)}$.

Solución:

$$\frac{\partial \mathcal{L}}{\partial \hat{\boldsymbol{y}}} = \frac{2}{N}(\hat{\boldsymbol{y}} - \boldsymbol{y})^t = (1.9640, 0.9982)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(3)}} = \mathbf{X}^{(3)t}\left(\frac{\partial \mathcal{L}}{\partial \hat{\boldsymbol{y}}}\right)^t = \begin{pmatrix} 0.9820 & 0.9991 \\ 0.9820 & 0.9991 \end{pmatrix}\begin{pmatrix} 1.9640 \\ 0.9982 \end{pmatrix} = \begin{pmatrix} 2.9259 \\ 2.9259 \end{pmatrix}$$

$$\mathbf{W}^{(3)t} = \mathbf{W}^{(3)t} - \eta\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(3)}} = \begin{pmatrix} 1 \\ 1 \end{pmatrix} - 0.1\begin{pmatrix} 2.9259 \\ 2.9259 \end{pmatrix} = \begin{pmatrix} 0.7074 \\ 0.7074 \end{pmatrix}$$

In [ ]:
```
import numpy as np; np.set_printoptions(precision=4); import keras
X1 = keras.Input(shape=(3,)); C1 = keras.initializers.Constant(1.)
X3 = keras.layers.Dense(2, activation='sigmoid', kernel_initializer=C1, bias_initializer=C1)(X1)
hy = keras.layers.Dense(1, activation=None, kernel_initializer=C1, bias_initializer=C1)(X3)
M = keras.Model(inputs=X1, outputs=hy);
```

In [ ]:
```
X = np.array([[1, 1, 1], [2, 2, 2]]); y = np.array([[1], [2]])
M_X3 = keras.Model(inputs=X1, outputs=M.layers[1].output); print('X3 =', M_X3(X))
print('hy =', M(X))
M.compile(loss='mean_squared_error', optimizer=keras.optimizers.SGD(learning_rate=0.1))
M.fit(X, y, epochs=1, verbose=1); print(M.layers[1].get_weights(), "\n", M.layers[2].get_weights())
```

```
X3 = tf.Tensor(
[[0.982  0.982 ]
 [0.9991 0.9991]], shape=(2, 2), dtype=float32)
hy = tf.Tensor(
[[2.964 ]
 [2.9982]], shape=(2, 1), dtype=float32)
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 256ms/step - loss: 2.4269
[array([[0.9963, 0.9963],
       [0.9963, 0.9963],
       [0.9963, 0.9963]], dtype=float32), array([0.9964, 0.9964], dtype=float32)]
 [array([[0.7074],
       [0.7074]], dtype=float32), array([0.7038], dtype=float32)]
```

5 Ejemplo para XOR con salida softmax

Problema. Sea el MLP para XOR

$$\boldsymbol{x}^{(1)} \in \{0,1\}^2 \;\rightarrow\; \boldsymbol{x}^{(2)} = \mathbf{W}^{(1)}\boldsymbol{x}^{(1)} + \boldsymbol{b}^{(1)} \in \mathbb{R}^2 \;\rightarrow\; \boldsymbol{x}^{(3)} = \mathrm{ReLU}(\boldsymbol{x}^{(2)}) \in \mathbb{R}^2$$
$$\rightarrow\; \boldsymbol{x}^{(4)} = \mathbf{W}^{(3)}\boldsymbol{x}^{(3)} + \boldsymbol{b}^{(3)} \in \mathbb{R}^2 \;\rightarrow\; \boldsymbol{x}^{(5)} = \hat{\boldsymbol{y}} = \mathcal{S}(\boldsymbol{x}^{(4)}) \in [0,1]^2$$

$$\mathbf{W}^{(1)} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \quad \boldsymbol{b}^{(1)} = \begin{pmatrix} -1 \\ 0.5 \end{pmatrix} \quad \mathbf{W}^{(3)} = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix} \quad \boldsymbol{b}^{(3)} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

Dado $\mathbf{X} = \begin{pmatrix} \boldsymbol{x}_1^t \\ \boldsymbol{x}_2^t \\ \boldsymbol{x}_3^t \\ \boldsymbol{x}_4^t \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}$ y $\mathbf{Y} = \begin{pmatrix} \boldsymbol{y}_1^t \\ \boldsymbol{y}_2^t \\ \boldsymbol{y}_3^t \\ \boldsymbol{y}_4^t \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}$, se está aplicando backprop para minimizar la entropía cruzada mediante una iteración de GD con $\eta = 0.1$. Se ha completado el paso forward:

$$\mathbf{X}^{(2)} = \begin{pmatrix} -1 & 0.5 \\ 0 & 1.5 \\ 0 & 1.5 \\ 1 & 2.5 \end{pmatrix} \;\rightarrow\; \mathbf{X}^{(3)} = \begin{pmatrix} 0 & 0.5 \\ 0 & 1.5 \\ 0 & 1.5 \\ 1 & 2.5 \end{pmatrix} \;\rightarrow\; \mathbf{X}^{(4)} = \begin{pmatrix} 0.5 & -0.5 \\ -0.5 & 0.5 \\ -0.5 & 0.5 \\ -0.5 & 0.5 \end{pmatrix}$$
$$\rightarrow\; \hat{\mathbf{Y}} = \mathbf{X}^{(5)} = \begin{pmatrix} 0.7311 & 0.2689 \\ 0.2689 & 0.7311 \\ 0.2689 & 0.7311 \\ 0.2689 & 0.7311 \end{pmatrix} \;\rightarrow\; \mathcal{L} = x^{(6)} = 0.5633$$

Aplica el paso backward (parcialmente) y actualiza $\boldsymbol{b}^{(3)}$.

Solución:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}^{(4)}} = \frac{1}{N}(\hat{\mathbf{Y}} - \mathbf{Y})^t = \frac{1}{4}\left(\begin{pmatrix} 0.7311 & 0.2689 \\ 0.2689 & 0.7311 \\ 0.2689 & 0.7311 \\ 0.2689 & 0.7311 \end{pmatrix} - \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}\right)^t = \begin{pmatrix} -0.0672 & 0.0672 & 0.0672 & -0.1828 \\ 0.0672 & -0.0672 & -0.0672 & 0.1828 \end{pmatrix}$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{b}^{(3)}} = \mathbf{1}^t \left(\frac{\partial \mathcal{L}}{\partial \mathbf{X}^{(4)}}\right)^t = (1,1,1,1) \begin{pmatrix} -0.0672 & 0.0672 \\ 0.0672 & -0.0672 \\ 0.0672 & -0.0672 \\ -0.1828 & 0.1828 \end{pmatrix} = (-0.1156, 0.1156)$$

$$\boldsymbol{b}^{(3)t} = \boldsymbol{b}^{(3)t} - \eta \frac{\partial \mathcal{L}}{\partial b^{(3)}} = (1,-1) - 0.1(-0.1156, 0.1156) = (1.0116, -1.0116)$$

In [ ]:
```
import numpy as np; np.set_printoptions(precision=4); import tensorflow as tf; import keras
X1 = keras.Input(shape=(2,))
W1 = tf.constant_initializer([[1, 1], [1, 1]]); b1 = tf.constant_initializer([[-1], [.5]])
X3 = keras.layers.Dense(2, activation='relu', kernel_initializer=W1, bias_initializer=b1)(X1)
W3 = tf.constant_initializer([[1, -1], [-1, 1]]); b3 = tf.constant_initializer([[1], [-1]])
hY = keras.layers.Dense(2, activation='softmax', kernel_initializer=W3, bias_initializer=b3)(X3)
M = keras.Model(inputs=X1, outputs=hY);
```

In [ ]:
```
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]]); y = np.array([[1, 0], [0, 1], [0, 1], [1, 0]])
M_X3 = keras.Model(inputs=X1, outputs=M.layers[1].output); print('X3 =', M_X3(X))
print('hY =', M(X))
M.compile(loss='crossentropy', optimizer=keras.optimizers.SGD(learning_rate=0.1))
M.fit(X, y, epochs=1, verbose=1); print(M.layers[1].get_weights(), "\n", M.layers[2].get_weights())
```

```
X3 = tf.Tensor(
[[0.  0.5]
 [0.  1.5]
 [0.  1.5]
 [1.  2.5]], shape=(4, 2), dtype=float32)
hY = tf.Tensor(
[[0.7311 0.2689]
 [0.2689 0.7311]
 [0.2689 0.7311]
 [0.2689 0.7311]], shape=(4, 2), dtype=float32)
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 278ms/step - loss: 0.5633
[array([[1.0366, 0.9769],
       [1.0366, 0.9769]], dtype=float32), array([-0.9634,  0.4769], dtype=float32)]
 [array([[ 1.0183, -1.0183],
       [-0.9711,  0.9711]], dtype=float32), array([ 1.0116, -1.0116], dtype=float32)]
```

6 Ejercicio: XOR con salida sigmoide

Problema. Sea el MLP para XOR

$$\boldsymbol{x}^{(1)} \in \{0,1\}^2 \;\rightarrow\; \boldsymbol{x}^{(2)} = \mathbf{W}^{(1)}\boldsymbol{x}^{(1)} + \boldsymbol{b}^{(1)} \in \mathbb{R}^2 \;\rightarrow\; \boldsymbol{x}^{(3)} = \mathrm{ReLU}(\boldsymbol{x}^{(2)}) \in \mathbb{R}^2$$
$$\rightarrow\; \boldsymbol{x}^{(4)} = \mathbf{W}^{(3)}\boldsymbol{x}^{(3)} + \boldsymbol{b}^{(3)} \in \mathbb{R}^2 \;\rightarrow\; x^{(5)} = \hat{y} = \sigma(x^{(4)}) \in [0,1]$$

$$\mathbf{W}^{(1)} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \quad \boldsymbol{b}^{(1)} = \begin{pmatrix} 0 \\ -1 \end{pmatrix} \quad \mathbf{W}^{(3)} = \begin{pmatrix} 1 & -1 \end{pmatrix} \quad b^{(3)} = 0$$

Dado $\mathbf{X} = \begin{pmatrix} \boldsymbol{x}_1^t \\ \boldsymbol{x}_2^t \\ \boldsymbol{x}_3^t \\ \boldsymbol{x}_4^t \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}$ y $\boldsymbol{y} = \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$, se está aplicando backprop para minimizar la entropía cruzada binaria mediante una iteración de GD con $\eta = 0.1$. Se ha completado el paso forward:

$$\mathbf{X}^{(2)} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix} \rightarrow \mathbf{X}^{(3)} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix} \rightarrow \mathbf{X}^{(4)} = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 1 \end{pmatrix} \rightarrow \hat{\boldsymbol{y}} = \mathbf{X}^{(5)} = \begin{pmatrix} 0.5000 \\ 0.7310 \\ 0.7310 \\ 0.7310 \end{pmatrix} \rightarrow \mathcal{L} = x^{(6)} = 0.6582$$

Aplica el paso backward (parcialmente) y actualiza $\mathbf{W}^{(1)}$.

Solución:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}^{(4)}} = \frac{1}{N}(\hat{\boldsymbol{y}} - \boldsymbol{y})^t = (0.1250, -0.0673, -0.0673, 0.1828)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}^{(3)}} = \mathbf{W}^{(3)t}\frac{\partial \mathcal{L}}{\partial \mathbf{X}^{(4)}} = \begin{pmatrix} 0.1250 & -0.0673 & -0.0673 & 0.1828 \\ -0.1250 & 0.0673 & 0.0673 & -0.1828 \end{pmatrix}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}^{(2)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{X}^{(3)}} \odot \mathrm{ReLU}'(\mathbf{X}^{(2)t}) = \begin{pmatrix} 0 & -0.0673 & -0.0673 & 0.1828 \\ 0 & 0 & 0 & -0.1828 \end{pmatrix}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \mathbf{X}^{(1)t}\left(\frac{\partial \mathcal{L}}{\partial \mathbf{X}^{(2)}}\right)^t = \begin{pmatrix} 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & 0 \\ -0.0673 & 0 \\ -0.0673 & 0 \\ 0.1828 & -0.1828 \end{pmatrix} = \begin{pmatrix} 0.1155 & -0.1828 \\ 0.1155 & -0.1828 \end{pmatrix}$$

$$\mathbf{W}^{(1)t} = \mathbf{W}^{(1)t} - \eta\,\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} - 0.1 \begin{pmatrix} 0.1155 & -0.1828 \\ 0.1155 & -0.1828 \end{pmatrix} = \begin{pmatrix} 0.9885 & 1.0183 \\ 0.9885 & 1.0183 \end{pmatrix}$$

In [ ]:
```
import numpy as np; np.set_printoptions(precision=4); import tensorflow as tf; import keras
X1 = keras.Input(shape=(2,))
W1 = tf.constant_initializer([[1, 1], [1, 1]]); b1 = tf.constant_initializer([[0], [-1]])
X3 = keras.layers.Dense(2, activation='relu', kernel_initializer=W1, bias_initializer=b1)(X1)
W3 = tf.constant_initializer([[1, -1]]); b3 = tf.constant_initializer([0])
hy = keras.layers.Dense(1, activation='sigmoid', kernel_initializer=W3, bias_initializer=b3)(X3)
M = keras.Model(inputs=X1, outputs=hy);
```

In [ ]:
```
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]]); y = np.array([[0], [1], [1], [0]])
M_X3 = keras.Model(inputs=X1, outputs=M.layers[1].output); print('X3 =', M_X3(X))
print('hy =', M(X))
M.compile(loss='crossentropy', optimizer=keras.optimizers.SGD(learning_rate=0.1))
M.fit(X, y, epochs=1, verbose=1); print(M.layers[1].get_weights(), "\n", M.layers[2].get_weights())
```

```
X3 = tf.Tensor(
[[0. 0.]
 [1. 0.]
 [1. 0.]
 [2. 1.]], shape=(4, 2), dtype=float32)
hy = tf.Tensor(
[[0.5   ]
 [0.7311]
 [0.7311]
 [0.7311]], shape=(4, 1), dtype=float32)
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 310ms/step - loss: 0.6582
[array([[0.9884, 1.0183],
       [0.9884, 1.0183]], dtype=float32), array([-0.0048, -0.9817], dtype=float32)]
 [array([[ 0.9769],
       [-1.0183]], dtype=float32), array([-0.0173], dtype=float32)]
```